

SOUTENANCE DE PROJET

PROJET PS5

ÉQUIPE O

PRÉSENTATION DU PROJET



+

Java

Maven

Digitaliser "Les aventuriers du rail - Allemagne" grâce à Java

OBJECTIFS DU PROJET

GitHub mockito Maven™

GESTION DE PROJET

- Exploitation des *Issues* et du *Branching*
- *Gestion des dépendances, construction du projet, documentation*
- *Gestion des comportements*



RECHERCHE ET CONCEPTION

- Exercice pour l'abstraction et la conception de projet
- Effort d'implémentation des intervenants/ éléments du jeu de la "bonne manière".
- Conserver une architecture cohérente et fonctionnelle
- Vision à long terme



ACQUISITION DE COMPÉTENCES

- Prise en main de plusieurs nouveau outils
- Meilleure connaissance des limites de notre projet
- Amélioration du niveau en Java
- Meilleure cohésion de groupe / meilleure communication

BILAN DES FONCTIONNALITÉS

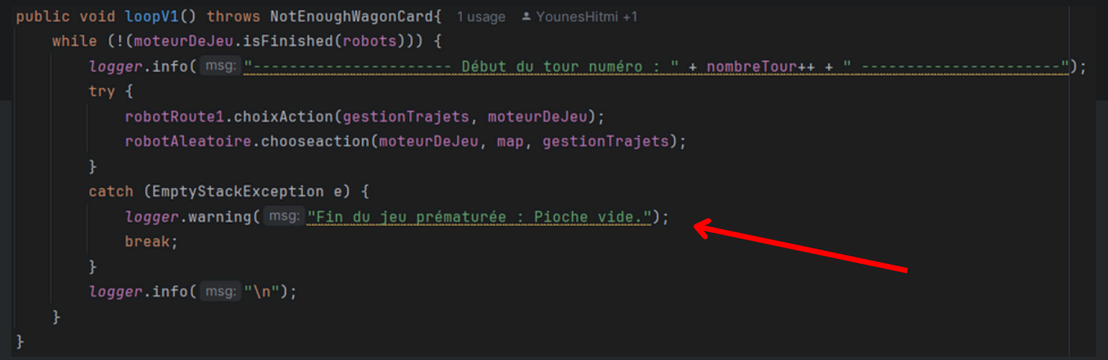
A ÉTÉ IMPLÉMENTÉ ✅

ÉLÉMENTS DU JEU 🎲
-> WAGONS <-
-> CARTES WAGONS/DESTINATIONS <-
-> MEEPLES <-
-> MOTEUR DE JEU <-
-> ROBOTS <-
-> PLATEAU <-

MÉCANIQUES PRINCIPALES 🔄
-> SYSTÈME DE PIOCHE <-
-> POSE DE WAGONS <-
-> RÉCUPÉRATION DES MEEPLES <-

CONDITIONS DE VICTOIRE 🏆
-> TITRE GLOBETROTTER <-
-> MAJORITÉ DE MEEPLES <-

DÉFAUTS D'IMPLÉMENTATION🚫

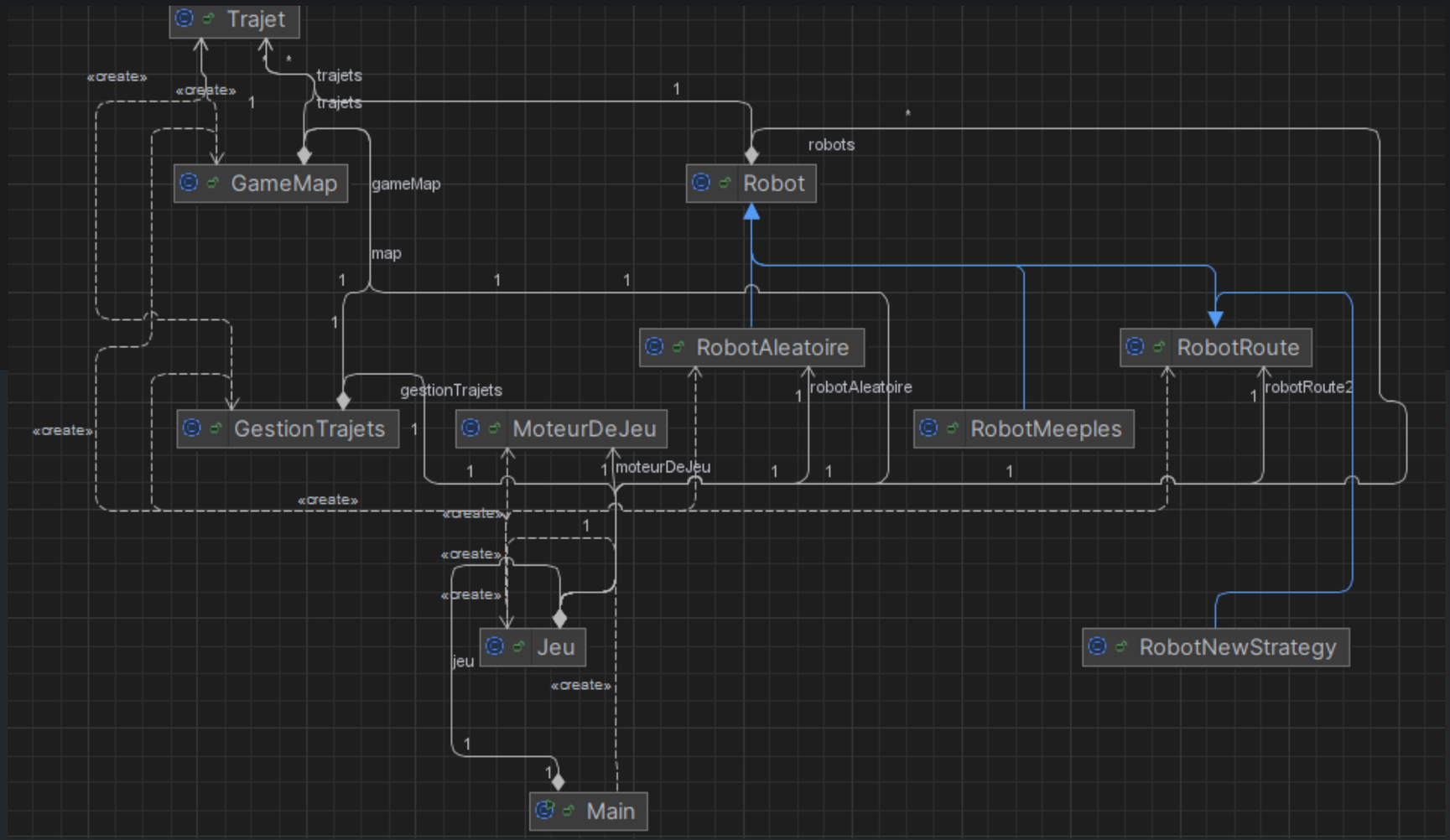


```
public void loopV1() throws NotEnoughWagonCard{  1 usage  YounesHitmi +1
    while (!(moteurDeJeu.isFinished(robots))) {
        logger.info( msg: "---------------------- Début du tour numéro : " + nombreTour++ + " ----------------------");
        try {
            robotRoute1.choixAction(gestionTrajets, moteurDeJeu);
            robotAleatoire.chooseaction(moteurDeJeu, map, gestionTrajets);
        }
        catch (EmptyStackException e) {
            logger.warning( msg: "Fin du jeu prématurée : Pioche vide.");
            break;
        }
        logger.info( msg: "\n");
    }
}
```

2. QUALITÉ DU CODE

Architecture / Sonar / Prise de recul sur le code

ARCHITECTURE GLOBALE



QUALITÉ DU CODE

```
public void poseTrain(Trajet trajet, MoteurDeJeu moteurDeJeu, GestionTrajets gestionTrajets) throws NotEnoughWagonCard
    moteurDeJeu.poseTrain( robot: this,trajet,gestionTrajets);
}

public void poseTrain(Robot robot, Trajet trajet,GestionTrajets gestionTrajets) throws NotEnoughWagonCard {
```



LES DEUX OBJETS SE CONNAISSENT

- ORGANISATION EN PACKAGE CLAIR

- BONNE STRUCTURE DU CODE ‘

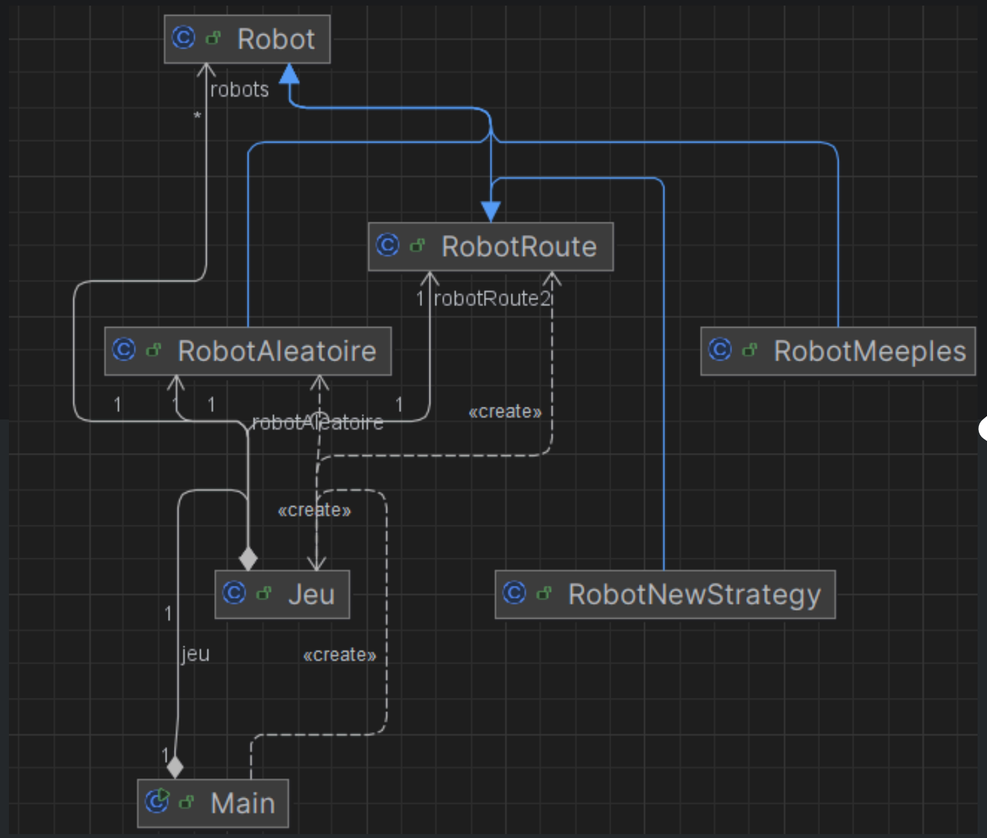
- PARTAGE DES RESPONSABILITÉS ENTRE LES CLASSES ET ABSENCE DE CLASSES “GOD”

NON QUALITÉ DU CODE

```
private Carte piocherCarte(Robot robot) {  2 usages   IsmailG +1
    Carte carte = getPioche().piocher();
    affichage.affichePiocher(robot, carte);
    robot.getWagonByCouleur().put(
            carte.getCouleur(),
            robot.getWagonByCouleur().get(carte.getCouleur()) + 1
    );
    return carte;
}
public void poseTrain(Robot robot, Trajet trajet,GestionTrajets gestionTrajets) throws NotEnoughWagonCard {
    Route route = trajet.getRoute();
    Couleur couleurRoute = determineRouteColor(robot, route);
    int nombreWagons = robot.getWagonByCouleur().getOrDefault(couleurRoute, defaultValue: 0);
    if (route.isClaimable(nombreWagons, new Wagon(couleurRoute)) || rainbowCase(robot, trajet)) {
        updateRobotWagonData(robot, route, trajet, couleurRoute);
        processRouteClaim(robot, trajet, route, couleurRoute);
        gestionTrajets.updateTrajetsDisponibles(trajet);
        robot.addNbRoute();
    } else {
        throw new NotEnoughWagonCard(robot + " n'a pas assez de carte wagon.");
    }
}
```

UTILISATION DE LA HASHMAP PLUTOT QUE DE LA MAIN DANS LA LOGIQUE DU JEU

NON QUALITÉ DU CODE

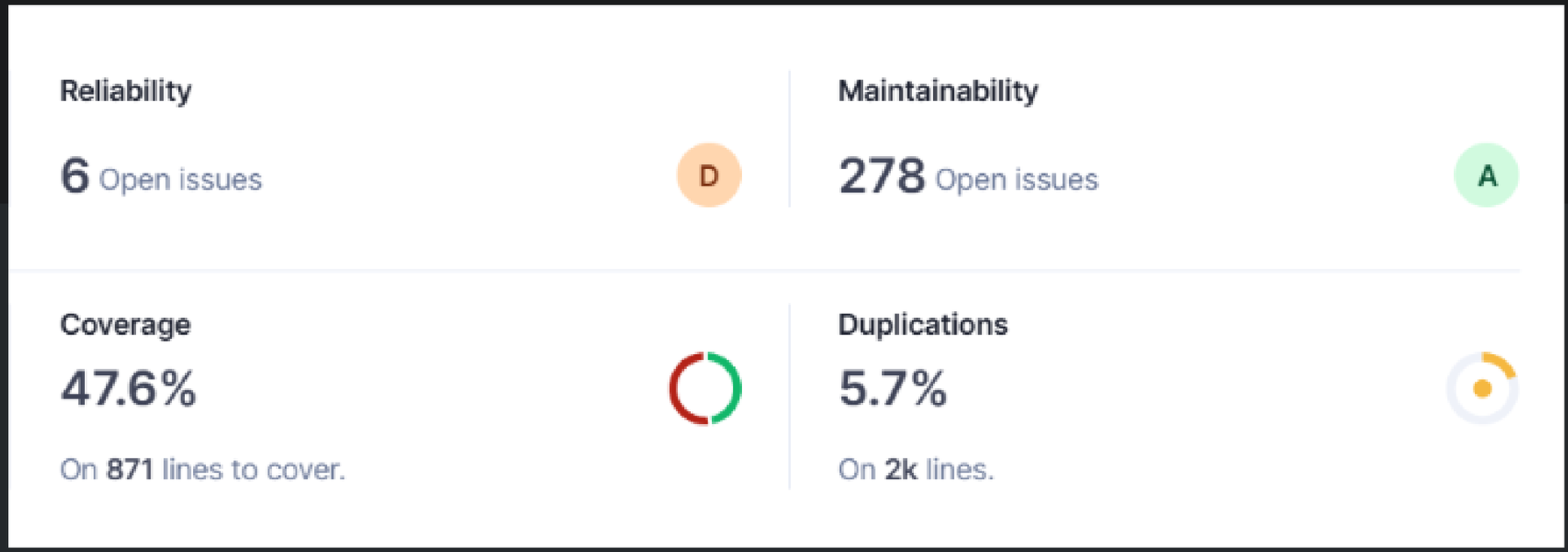


NON UTILISATION D'UNE CLASSE ABSTRAITE

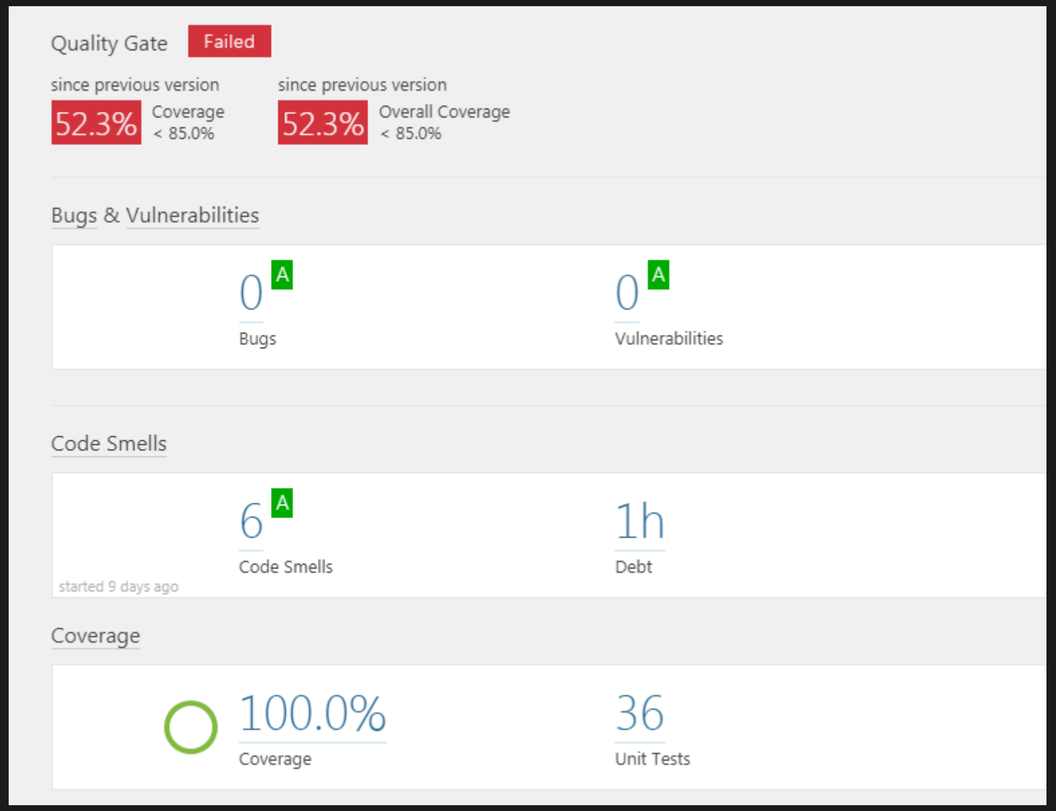
3. CONFIANCES AUX CODES

SONAR - PARTIE CONFIANCE / PAS CONFIANCE - TEST

SONAR


Reliability
6 Open issues
D
Maintainability
278 Open issues
A
Coverage
47.6%
On 871 lines to cover.
Duplications
5.7%
On 2k lines.

EN QUOI C'EST IMPORTANT ?



- Confiance au code = Fiabilité et stabilité
- Suivi du coverage
- Prendre des bonnes habitudes
- Une bonne maintenabilité du code
- Un feedback régulier

PARTIE CONFIANCE

- Le moteur de jeu
- Robot
- Gestionnaire
- Utilitaire

MAIS, Pourquoi ?

src/main/java/fr/cotedazur/univ/polytech/teamO/pla.../**Route.java**	85.3%	0	5
src/main/java/fr/cotedazur/univ/polytech/teamO/ges.../**Score.java**	85.7%	1	86
src/main/java/fr/cotedazur/univ/polytech/teamO/pla.../**Ville.java**	85.9%	9	2
src/main/java/fr/cotedazur/univ/polytech/teamO/pla.../**Trajet.java**	86.7%	0	4
src/main/java/fr/cotedazur/univ/polytech/teamO/bot/**Robot.java**	86.9%	8	3
src/main/java/fr/cotedazur/univ/polytech/teamO/bot/**RobotRoute.java**	87.1%	9	4

src/main/java/fr/cotedazur/univ/polytech/teamO/pio.../**CarteWagon.java**	77.8%	1	1
src/main/java/fr/cotedazur/univ/polytech/teamO/pla.../**GameMap.java**	79.1%	26	14
src/main/java/fr/cotedazur/univ/polytech/teamO/ges.../**MoteurDeJeu.java**	79.1%	25	20
src/main/java/fr/cotedazur/univ/polytech/teamO/pio.../**Carte.java**	83.3%	1	2
src/main/java/fr/cotedazur/univ/polytech/teamO/ges.../**GestionEtatJeu.java**	83.8%	13	6

src/main/java/fr/cotedazur/univ/polytech/teamO/**Commands.java**	0.0%	5	0
src/main/java/fr/cotedazur/univ/polytech/teamO/aut.../**csv.java**	0.0%	35	8
src/main/java/fr/cotedazur/univ/polytech/teamO/cla.../**EmptyListException.java**	0.0%	2	0
src/main/java/fr/cotedazur/univ/polytech/teamO/aut.../**Jeu.java**	0.0%	129	30
src/main/java/fr/cotedazur/univ/polytech/teamO/**Main.java**	0.0%	52	22
src/main/java/fr/cotedazur/univ/polytech/teamO/**NotValidCommand.java**	0.0%	2	0

```
public PiocheDestinationCourte getPiocheDesinationCourte() {
    return piocheDesinationCourte;
}
```

```
    @Override
    public String toString() {
        String str = "";
        for(T carte : mainJoueur){
            str += carte.toString() + "\n";
        }
        return str;
    }
}
```

```java
*/
public Carte piocherSurTable(Robot robot,Carte carte) throws NullPointerException{
    try{
        Carte cartePiochee=getPiocheSurTable().piocher(carte);
        affichage.affichePiocherSurTable(robot,cartePiochee);
        try {
            getPiocheSurTable().addCarte(pioche.piocher());
        }
        catch(EmptyStackException e) {
            replacePiocheByDefausse();
            getPiocheSurTable().addCarte(pioche.piocher());
        }
        updateRobotPiocheSurTable(robot,cartePiochee);
        return cartePiochee;}
    catch(NullPointerException e){
        throw e;
    }
}
```

PARTIE PAS EN CONFIANCE

POURQUOI ?

- Des test pas assez poussés dans des cas précis
- Pas assez de cas limites MAIS on y a pensé

ÉVOLUTION DU COVERAGE

Reliability
6 Open issues D

Maintainability
278 Open issues A

Coverage
47.6%
On **871** lines to cover.

Duplications
5.7%
On **2k** lines.

Reliability
0 Open issues A

Maintainability
340 Open issues A

Coverage
73.5%
On **1.5k** lines to cover.

Duplications
2.4%
On **3.8k** lines.

Reliability
9 Open issues D

Maintainability
287 Open issues A

Coverage
63.5%
On **1.1k** lines to cover.

Duplications
0.0%
On **2.4k** lines.

TESTS ET DÉMONSTRATIONS

4. RÉFLEXION SUR LE PROJET

ORGANISATION/ AMÉLIORATIONS/ BILAN DU PROJET

ORGANISATION

TRAVAIL

- USER STORY → En tant que Bot,
  Je veux piocher deux cartes,
  Afin de pour pouvoir jouer.
- TASK
- RÉPARTITION DU TRAVAIL

	YOUNES	ADAM	NICOLAS	ISMAIL
Robots	✖	✖	✖	✖
CSV		✖		
Logger	✖			
Utilitaire	✖	✖		
Moteur de jeu et Gestion de trajets			✖	✖

CODE

- Maven™

AMÉLIORATIONS

SONAR
MOCKITO

GITHUB

COMMUNICATION



QUALITÉ DU CODE

DOJO POKER

BILAN DU PROJET

SI C'ÉTAIT À REFAIRE ....



INVESTISSEMENT

COMPÉTENCES ACQUISES EN POO

LIVRAISON D'UN PROJET FONCTIONNEL



MAUVAISE GESTION DE LA CHARGE DE TRAVAIL DANS LE TEMPS

MÉTHODE DE BRANCHING

USER STORY EN DÉBUT DE PROJET

MERCI !